Data Structures
Lab Sheet 1
Engineering - Instituto Superior de Engenharia de Coimbra

1 - For each of the following programs:

- Perform analysis of complexity. Estimate the increase in execution time if the size N is increased 4x.

```
a) is (long i = 0; i <n; i ++)
          for (long j = 0; j <N; j ++)
               sum ++;

b) is (long i = 0; i <n; i ++)
          sum ++;

c) is (long i = 0; i <n; i + = 2)
          sum ++;

d) for (long i = 0; i <1000; i ++)
          for (long j = 0; j <N; j ++)
               sum ++;

e) for (long i = 0; i <n; i ++)
          sum ++;
     for (long j = 0; j <N; j ++)
          sum ++;

f) if (n> 20,000) n = 20,000;
     for (long i = 0; i <n; i ++)
          for (long j = 0; j <N; j ++)
               sum ++;


g) for (long i = 0; i <n; i ++)
          for (long j = 0; j <n * n; j ++)
               sum ++;

h) is (long i = 0; i <n; i ++)
          for (long j = 0; j <i; j ++)
               sum ++;
```

```
i) for (long i = 0; i <n; i ++)
          for (long j = 0; j <n * n; j ++)
               for (long k = 0; k <j; k ++)
                    sum ++;

j) for (long I = 1; i <n; i + = 2)
          sum ++;
```

2 - For each of the lines a) –j) of the previous question, implement the code and check the runtime (use System.nanoTime () to get a long value to the current time in ns (1 ns = 10-9 seconds)). Compare your predictions with the results. Should seek to find values for N that result in significant run times (a few seconds). You can set a constant value to the constant long adding an L; for example n = 127343734L long.
Depending on your computer, it may be difficult to fulfill it for some lines.

3 - Implement and test the three maximum contiguous sequence calculation algorithms studied in the lectures. Analyze their performance and make sure it is aligned with the respective class of complexity.

4 - Develop, analyze, and implement an algorithm that checks for an integer i in position i in a specific ordered array of integers (no repetitions). What is the complexity of the algorithm? (Hint: it is possible to do better than O (N)).

5 - Consider an NxN matrix, in which the numbers are arranged in ascending order, both in rows and in columns. Consider an algorithm that checks whether a number is or is not in the array:
a) Create, implement and test one algorithm with complexity of O (N^2)
b) create, deploy and test a O(log N) complexity algorithm

6 - A majority element of an N dimensional array is a value that appears more than N / 2 times. For example, the majority element of [3,3,4,2,4,4,2,4,4] is 4, while [3,3,4,2,4,4,2,4] does not a majority element. Build, analyze and implement an algorithm to solve this problem. (Challenge: It is possible (but not too obvious), solve the problem in linear time ...)